

ESG Scope 1/2/3 RAG Assistant

Interview preparation guide — what I built, why I built it that way, and how to talk about it.

Source data	7 corporate sustainability reports, 591 pages
Indexed	2,833 chunks in ChromaDB, built in 66 seconds
Stack	Python, PyMuPDF, sentence-transformers, ChromaDB, Google Gemini, Streamlit
Interfaces	Streamlit web app and a command-line tool
Repository	github.com/gunaprakshkethineni/esg-rag-assistant

1. What the project is

THE ONE-PARAGRAPH VERSION

I built a retrieval-augmented generation (RAG) chatbot that answers questions about corporate carbon emissions by reading the companies' own sustainability reports. You ask in plain English, and it answers with the exact page number the figure came from. The part I care most about is that it doesn't just trust the model: two independent checks run in code after every answer to verify the citation is real and the number actually appears in the retrieved text.

SAY THIS IN THE FIRST 30 SECONDS

"I built a RAG chatbot over seven real sustainability reports, about 600 pages. It answers with the exact source page. The interesting part isn't the LLM call — it's that I didn't trust it. I built two verification checks and measured everything with a hand-built test set instead of just claiming it works."

2. Why — the problem it solves

LEAD WITH THIS, NOT THE TECH STACK

Sustainability reports are long, dense with numbers, and inconsistent. An analyst who needs one figure has to search hundreds of pages, then repeat that for every company they cover.

But the reason this is genuinely hard isn't length — it's that **the same fact often has several different, equally correct answers**. In my own data, Siemens' Scope 3 emissions appear as **416,758 thousand metric tons** in one report and **178,835 ktCO₂e** in another. Both are right. They cover different reporting boundaries: one counts the full downstream lifecycle of every product sold, the other only the subset tracked against reduction targets. Nothing in the document shouts this at you. Pick the wrong one and you misreport a company's footprint by more than double.

That is why I made the page citation the core feature rather than a nice-to-have.

Who actually has this problem:

- ESG analysts at asset managers, scoring companies before investing
- ESG rating agencies, whose whole business is reading these reports
- Sustainability consultants building competitor benchmarks
- Compliance teams facing EU CSRD reporting obligations
- Journalists and NGOs checking marketing claims against the fine print

Why not just paste it into ChatGPT:

- **No traceability.** An unsourced number is useless in compliance-adjacent work.
- **Scale.** Needle-in-haystack accuracy degrades badly over hundreds of pages.
- **Hallucination is a dealbreaker here**, not an annoyance. An invented emissions figure in an investment memo is a legal and reputational problem.

3. How it works

THE PIPELINE IN FOUR STAGES

Stage	What happens
1. Ingest run once	PyMuPDF reads each PDF page by page , not as one blob. Text is split into ~1,000 character chunks with 150 characters of overlap, and a chunk never crosses a page boundary — that is what makes exact page citation possible. Each chunk is embedded locally with MiniLM (384 dimensions) and stored in ChromaDB with metadata: company, report, year, page.
2. Retrieve per question	The question is embedded with the same model. ChromaDB returns a wide candidate pool, which is then re-ranked by fusing two signals — semantic similarity and keyword matching — using Reciprocal Rank Fusion, plus a bonus for chunks containing every query word. Neighbouring chunks from the same page get pulled in so a fact split across a boundary stays intact.
3. Generate per question	Retrieved chunks are labelled with company, year and page, then sent to Gemini with a strict instruction: answer only from these excerpts, cite every claim as [Company, Year, p. X], and if the answer isn't here, return a fixed refusal sentence instead of guessing.
4. Verify per answer	Citation check: every page number cited must be one that was actually retrieved. Grounding check: every figure stated must appear verbatim in the retrieved text. Both results are shown to the user as badges, not hidden.

4. Tools I used and why

EXPECT TO BE ASKED TO DEFEND EVERY ONE OF THESE

Tool	Why I chose it	What I rejected, and why
PyMuPDF PDF parsing	Fast, pure-Python wheel, no system dependencies. Extracts text per page, which my citation design depends on.	pdfplumber is better at table structure but noticeably slower over 591 pages. PyPDF2 has weaker text extraction. Poppler/pdftotext needs a system binary install, which breaks portability.
MiniLM (all-MiniLM-L6-v2) embeddings	Runs locally on CPU. I embed 2,833 chunks at ingest and once per query, so this is the highest-volume operation in the system. Local means no cost, no rate limits, and no network hop during retrieval.	OpenAI/Gemini embedding APIs would mean thousands of calls, real cost, and rate-limit handling — for a task a 90MB local model does well. Larger models (BGE, E5) score higher on benchmarks but are slower and my accuracy bottleneck turned out to be retrieval logic, not embedding quality.
ChromaDB vector store	Zero infrastructure, persists to disk, and has metadata filtering built in — which I use for the company filter. Right-sized for 2,833 chunks.	Pinecone/Weaviate are hosted services; unnecessary cost and setup at this scale. FAISS is faster but has no metadata filtering, which I need. pgvector means running Postgres for a single-user project.

Tool	Why I chose it	What I rejected, and why
Google Gemini generation	Generous free tier, and generation happens once per question so a strong model is worth the API call here. The SDK lets me list available models at runtime rather than hardcoding one.	OpenAI would work equally well — this was availability, not a technical argument, and I'd say so. A local LLM (Llama/Mistral) would remove the API dependency but needs a GPU for acceptable latency and I measured that generation quality mattered for citations.
Streamlit interface	Turns a Python function into a usable web app in a few dozen lines. The point of the project is the pipeline, not the frontend.	FastAPI + React means building and hosting two things. Gradio is comparable; Streamlit had better multi-tab layout for what I needed.
RAG overall approach	I need a citable page for every figure, and reports are republished annually — re-ingesting takes 66 seconds.	Fine-tuning teaches style, not verifiable facts, and a fine-tuned model can't tell you which page a number came from. Long-context prompting (dumping all 591 pages in) degrades badly on needle-in-haystack retrieval and gives no provenance.

5. The engineering story

THE STRONGEST THING YOU CAN TALK ABOUT

The final architecture wasn't designed up front. It's the result of specific failures I found by testing with deliberately sloppy, lowercase questions instead of the tidy sentences in my test set. These three bugs are worth memorising — they're what separate this from a tutorial project.

#	The bug	How I found and fixed it
1	Keyword matching could only re-rank, never recall	My keyword score was applied only to chunks the vector search already returned. I printed the ranks and found the correct chunk sitting at position 84 — it was never in the pool, so no amount of re-ranking could rescue it. Fixed by widening the candidate pool to 20x.
2	Substring matching created false keyword hits	I was checking <code>if keyword in text</code> , so “net” matched inside “internet” and “network”. Pages about IT infrastructure scored a perfect match on a net-zero question and beat the real page on the tiebreak. Fixed with whole-word matching that also tolerates spacing, so a user's “re100” matches the report's “RE 100”.
3	Equal rank fusion buried exact matches	A chunk containing every word of a short query still lost to chunks that merely ranked well semantically. Fixed with a coverage bonus, so matching the whole query counts on its own.

There is a fourth story, and it's the best one:

While clicking around my own app I got this answer: “*Siemens Energy's Scope 1 emissions were 175 (1,000 metric tons CO2e) [Siemens Energy, FY2024, p. 8]*”. The number was correct and the citation looked perfect. But when I expanded the sources panel, **page 8 was never retrieved**, and the number 175 didn't appear anywhere in the 16 chunks the model was shown. It answered from its own training data and attached a citation pointing at nothing.

That is why there are two checks rather than one. Citation-format checking cannot catch a fabricated number attached to a real page — so I added the grounding check that verifies every figure appears verbatim in the retrieved text.

6. Measured results

QUOTE THESE EXACTLY, DO NOT ROUND THEM UP

From a 26-question ground-truth set I wrote by reading the PDFs myself, including deliberate cases where the correct behaviour is to refuse.

Metric	Before my retrieval fixes	After
Retrieval accuracy	78.3%	91.3%
Answer accuracy	91.3%	95.7%
Citation consistency	57.7%	69.2%
Numeric grounding	76.9%	80.8%
Mean latency	1.05s	1.52s
Responses under 2 seconds	96.2%	96.2%

DO NOT OVERCLAIM

Never say 95% retrieval or perfect citations. 91.3% with a known failure mode is a far stronger answer than a round number you can't defend. The moment you quote something you didn't measure, the conversation becomes about whether you tested anything at all.

Why it isn't higher — have these three reasons ready:

- **Dense table pages embed poorly.** A page with 20+ unrelated metrics averages into a vague vector. I improved this but didn't fully solve it.
- **My grounding check is a strict verbatim match**, so it flags false negatives when the model writes "96.5%" and the report says "96.50%". True grounding is likely higher than 80.8% measures.
- **Some data exists only inside chart images** with no text layer, so it's genuinely unreachable without OCR.

7. Questions you will get

AND HOW TO ANSWER THEM

Why RAG instead of fine-tuning a model?

Fine-tuning teaches a model style and format, not verifiable facts. I need a citable page for every figure, and a fine-tuned model can't tell you where a number came from. These reports are also republished annually — re-ingesting takes 66 seconds, retraining doesn't.

Why ChromaDB specifically?

Local, persistent, no infrastructure, and metadata filtering built in — which I use for the company filter. At 2,833 chunks it's the right tool. At millions of vectors across tenants I'd move to pgvector or a managed service; I wouldn't pick Chroma for that.

Why local embeddings but a cloud LLM?

Different cost profiles. Embedding runs thousands of times — once per chunk at ingest, once per query — so a local model is free, rate-limit-free, and keeps retrieval off the network. Generation happens once per question and genuinely benefits from a strong model, so that's where the API call earns its cost.

How do you stop it hallucinating?

Three layers. The prompt restricts it to the provided excerpts with a fixed refusal string. The citation check verifies every cited page was actually retrieved. The grounding check verifies every figure appears verbatim in the retrieved text. The third exists because I found a case the second couldn't catch.

How do you know it actually works?

A 26-question ground-truth set I built by reading the PDFs myself. It measures retrieval accuracy, answer accuracy, refusal accuracy, both checks, and latency — and includes negative cases where the right answer is “I can't find that.” Every number I quote comes from that harness and it re-runs on demand.

Isn't this just a wrapper around an LLM API?

The API call is about fifteen lines. The work is the hybrid retrieval, the verification layer and the evaluation harness. I can show you what happens without them: retrieval drops from 91% to 78% and citation reliability from 69% to 58%. Those are measured, not hypothetical.

What was the hardest bug?

The one where keyword matching could only re-rank, never recall. The symptom looked like the model being dumb — it kept refusing questions I knew were answerable. The cause was three layers down: the right chunk sat at rank 84 and my keyword boost never saw it. Finding it meant instrumenting the ranking itself rather than trusting end-to-end output.

How would you scale this to a thousand companies?

Ingestion is embarrassingly parallel, so that just needs workers. The vector store moves to a managed service with metadata partitioning so candidate pools stay small per company. A paid API tier removes the 15-requests-per-minute ceiling that shaped several current decisions. Then caching for repeat questions and async generation instead of synchronous requests.

What's the DEGREE part?

A genuine discovery. While reading the Siemens report to write test questions, I found they publish their own framework — Decarbonization, Ethics, Governance, Resource efficiency, Equity, Employability — each with numeric targets. So instead of inventing a rubric, I use the pipeline to fact-check Siemens against their own promises. It found they beat their female-leadership target early but are behind on injury-rate reduction.

Where does it fail?

Cross-company questions with no filter are measurably less reliable. Chart-only data is invisible without OCR. The eval set is 26 questions — enough for patterns, not tight confidence intervals. And it isn't audit-grade: it's a research accelerator for someone who knows how to verify a number, not an autonomous fact-checker.

What would you do differently?

Build the evaluation harness first, not after. Almost every real bug came from measuring, and those took minutes to find, while the ones I found by eyeballing output took hours.

Did you use AI to build it?

Answer honestly and move to substance: yes, and here's what I learned. Then pivot straight to a bug you found and fixed — the substring bug is perfect for this. Being able to explain why RRF_K is 20 instead of the standard 60 proves ownership far better than a denial that can unravel.

8. How to demo the GitHub repo

THE ORDER TO CLICK THROUGH

Don't start at the file list — it's the least interesting view. Walk them through in this order, and let each step set up the next.

	Where	What to say
1	README ~30 seconds	Open with the problem , not the stack: analysts lose hours finding one number in 600 pages, and the same number often has two correct answers. Then point at the measured results table — it signals immediately that you tested this.
2	notebooks/ esg_rag_walkthrough .ipynb the main event, 3-4 min	This is your strongest asset — it renders on GitHub with screenshots and outputs. Scroll to section 5, “Where it first went wrong” and walk through the three bugs in order. Show the printed rank-84 output and the substring bug. This is the part that proves you engineered rather than assembled.
3	Section 8 of the notebook 1 min	The hallucination you nearly missed — the perfect-looking citation pointing at a page that was never retrieved. Then show the two checks catching it. Pair this with the screenshot of the app refusing to answer the Infosys question.
4	src/retriever.py 1 min	Only if they want code. Point at the coverage bonus and the whole-word matching, and explain why each line exists. The comments already tell that story.
5	eval/eval_report.md 1 min	Per-question results. Say the honest version: 91.3%, not 95%, and here's exactly why. This usually lands better than a perfect score would.
6	Live app if you have a laptop	Ask one question that works and one that must be refused, back to back. That contrast is the whole project in ten seconds. Run <code>streamlit run app.py</code> beforehand — first load takes ~30 seconds.

IF THEY ONLY GIVE YOU 60 SECONDS

README problem statement → notebook section 6 (the three bugs) → the honest accuracy number. Skip everything else. Those three beats tell a complete story: real problem, real debugging, real measurement.

9. Numbers worth memorising

SPECIFICS BEAT ROUND CLAIMS

Source reports	7	Retrieval accuracy	91.3%
Total pages	591	Answer accuracy	95.7%
Indexed chunks	2,833	Citation consistency	69.2%
Ingest time	66 seconds	Numeric grounding	80.8%
Embedding dimensions	384	Mean latency	1.52s
Eval questions	26	Under 2 seconds	96.2%

One fact worth keeping in your pocket: **Microsoft's own report shows their overall emissions up 29.1%** against baseline, despite the public “carbon negative by 2030” pledge. It's a memorable, sourced example of what the tool surfaces.